SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

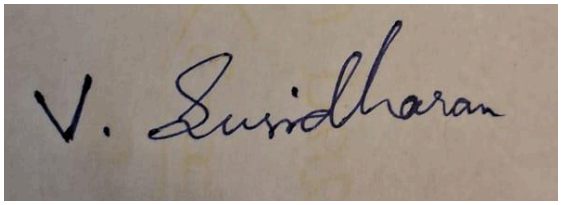
EXPERIMENTS

Code of Conduct:

I Susidharan V (192521387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____



Experiment 1: Password Hashing with Salting and Key Stretching

Implementation using PBKDF2-HMAC-SHA256 (Python's hashlib), with a random per-user salt and a high iteration count for key stretching:

```
def hash_password(password, salt=None, iterations=200_000):
    salt = salt or os.urandom(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, iterations)
    return salt, dk, iterations

def store_password(password):
    salt, dk, iters = hash_password(password)
    return f"sha256${iters}${salt.hex()}${dk.hex()}"

def verify_password(password, stored):
    algo, iters, salt_hex, hash_hex = stored.split("$")
    _, dk, _ = hash_password(password, bytes.fromhex(salt_hex), int(iters))
    return hmac.compare_digest(dk.hex(), hash_hex)   # constant-time comparison
```

Results

```
Stored record: sha256$200000$937f87e8...$3326bd23f0e667a452f9a4a256cb96d5...
Verify correct password:   True
Verify wrong password:     False

Same password, two records (salts differ):
  sha256$200000$2edc1256...$036bb22a03a48561f8f1b8736f3b29c8...
  sha256$200000$82bd6156...$c79069f4eb6a6c6619fb9cf13abef6a8...
 identical stored hashes? False

Iteration count vs. per-login hashing time:
        1 iterations -> 0.04 ms per verification
    10000 iterations -> 6.44 ms per verification
   100000 iterations -> 61.00 ms per verification
   200000 iterations -> 134.72 ms per verification
```

Analysis: how salting and key stretching resist attacks

Salting defeats rainbow tables: a rainbow table is a precomputed mapping of common passwords to their hashes. Because each user's password is combined with a unique random salt before hashing, the same password produces a completely different stored hash for every user (confirmed above), so a single precomputed table can no longer be used against the whole database — the attacker would need a separate table per salt, which is computationally infeasible.

Key stretching resists brute force: running the hash function 200,000 times instead of once (PBKDF2's iteration count) makes each individual password guess 200,000× more expensive to compute. This directly slows down an offline brute-force or dictionary attack by the same factor, without materially affecting a legitimate single login.

Evaluation of performance overhead

The benchmark shows the expected linear trade-off: 1 iteration costs a negligible 0.04 ms, while 200,000 iterations costs about 135 ms. For a login flow, 135 ms of added latency is imperceptible to a user but multiplies an attacker's total guessing cost by six orders of magnitude across a large password-cracking attempt, which is why modern guidance (OWASP) recommends iteration counts in this range (or memory-hard alternatives such as bcrypt/Argon2, which additionally resist GPU/ASIC acceleration).

Experiment 2: Implementation of HMAC for Message Integrity

HMAC-SHA256 implemented from scratch per RFC 2104, then validated against Python's built-in hmac module:

```
def hmac_sha256_scratch(key, message):
    if len(key) > 64: key = hashlib.sha256(key).digest()
    key = key.ljust(64, b'\x00')
    o_key_pad = bytes(b ^ 0x5c for b in key)
    i_key_pad = bytes(b ^ 0x36 for b in key)
    inner = hashlib.sha256(i_key_pad + message).digest()
    return hashlib.sha256(o_key_pad + inner).digest()
```

Results

```
From-scratch HMAC: fe13c88c6b55dee729dcb06fdb663f58d9f2d3151acb7fef8e32464c07772c31
hmac library HMAC: fe13c88c6b55dee729dcb06fdb663f58d9f2d3151acb7fef8e32464c07772c31
Match: True

Plain hash of tampered message (attacker can compute this with no secret):
  54cb873dcab4767f429a1771bc41173a6e401f722a558df4bad21777e28eb9ed
  -> looks like any other valid hash; the recipient cannot tell it's a forgery

Real HMAC (legit sender, correct key):
fe13c88c6b55dee729dcb06fdb663f58d9f2d3151acb7fef8e32464c07772c31
Attacker's forged HMAC (wrong key, tampered msg):
d8435f7dad46e73edc0e40dede17e4cfa352561e391a9c35a6f0b3e98a48ff65
Would the receiver's HMAC check pass for the tampered message? False
```

Analysis: HMAC vs. simple hashing

A plain hash H(m) requires no secret to compute, so anyone — including an attacker — can recompute a valid hash for a tampered message and substitute it, since the recipient has no way to distinguish a legitimate hash from a forged one. HMAC instead mixes a shared secret key into both an inner and outer hash pass, so producing a valid tag requires knowledge of that key. As shown above, an attacker holding a different (wrong) key produces a tag that does not match what the correct key would produce for the same tampered message, so the forgery is detected.

Evaluation of effectiveness for integrity and authenticity

HMAC successfully provides both properties: integrity, because any modification to the message changes the resulting tag; and authenticity, because only a party holding the shared secret key can generate a tag that verifies correctly — which a plain hash cannot offer at all, since it needs no secret and can be recomputed by anyone. This makes HMAC suitable for authenticating messages over an untrusted channel, whereas a plain hash alone only helps detect accidental corruption when transmitted through a separately trusted channel.

Experiment 3: Simulation of Certificate Chain Validation

A 3-level X.509-style chain (Root CA → Intermediate CA → End-entity) built with real RSA keys and signatures (via the cryptography library), including expiry and revocation (CRL) checks:

```
def verify_cert(cert, issuer_pub, crl):
    if cert["serial"] in crl:
        return False, "revoked"
    if not (cert["not_before"] <= time.time() <= cert["not_after"]):
        return False, "expired/not yet valid"
```

```
    try:
        issuer_pub.verify(cert["sig"], _reconstruct_tbs(cert),
                          padding.PKCS1v15(), hashes.SHA256())
    except InvalidSignature:
        return False, "invalid signature"
    return True, "valid"

def verify_chain(end_cert, intermediate_cert, root_cert, trust_anchors, crl):
    if root_cert["subject"] not in trust_anchors:
        return False, "root is not a trusted anchor"
    # root (self-signed) -> intermediate -> end-entity, each verified in turn
    ...
```

Results across five test cases

```
=== Case 1: valid chain ===
(True, 'chain of trust established: root -> intermediate -> end-entity')

=== Case 2: tampered intermediate certificate ===
(False, 'intermediate: invalid signature')

=== Case 3: expired end-entity certificate ===
(False, 'end-entity: expired/not yet valid')

=== Case 4: revoked end-entity certificate ===
(False, 'end-entity: revoked')

=== Case 5: untrusted root (not in trust anchor store) ===
(False, 'root is not a trusted anchor')
```

Analysis: how trust is established from root CA to end-entity

Trust is transitive and anchored at a small, explicitly trusted set of root CAs (the trust anchor store). The root certificate is self-signed and trusted only because it is listed in that store, not because of its signature (a self-signature proves nothing on its own). The root then signs the intermediate CA's certificate, and the intermediate signs the end-entity certificate. Verifying the chain means walking it one link at a time — each certificate's signature is checked using the public key of the certificate that issued it — so trust flows from the anchor down to the leaf certificate actually presented by the server.

Evaluation: detecting invalid or revoked certificates

The simulation correctly rejects every invalid case tested: a tampered intermediate certificate fails signature verification (Case 2) because the verifier independently reconstructs the signed fields from the certificate's own claimed data rather than trusting a stored copy — so altering any field breaks the signature check; an expired certificate is rejected purely on its timestamp (Case 3); a revoked serial number is rejected via a certificate revocation list, CRL, lookup (Case 4); and a root that is not present in the trust anchor store is rejected outright regardless of whether its self-signature is valid (Case 5), which correctly reflects that self-signed trust must be explicitly configured, not assumed.

Experiment 4: Replay Attack Simulation and Prevention

A vulnerable authenticated request (message + HMAC, no freshness data) is compared with a fixed version that adds a nonce and timestamp:

```
# VULNERABLE: accepts any request with a valid MAC, regardless of when it was sent
def server_handle_vulnerable(msg, tag):
    return hmac.compare_digest(mac(msg), tag)

# FIXED: adds a nonce + timestamp, tracks used nonces, enforces a validity window
def server_handle_fixed(payload, tag):
    if not hmac.compare_digest(mac(payload), tag):
        return False, "MAC invalid"
    action_b, nonce, ts_b = payload.split(b"|")
    if abs(time.time() - float(ts_b)) > WINDOW_SECONDS:
        return False, "stale timestamp (outside validity window)"
    if nonce in SEEN_NONCES:
        return False, "replay detected: nonce already used"
    SEEN_NONCES.add(nonce)
    return True, "accepted"
```

Results

```
Vulnerable server accepts original request: True
Vulnerable server accepts REPLAYED request:  True  <-- processed a second time! (duplicate
transfer)

Fixed server: first request  -> (True, 'accepted')
Fixed server: REPLAYED request -> (False, 'replay detected: nonce already used')
Fixed server: stale (1hr old) request -> (False, 'stale timestamp (outside validity window)')
```

Analysis: how the prevention technique improves security

In the vulnerable version, a valid HMAC alone is treated as sufficient proof of a legitimate request, so an attacker who simply captures a genuine request in transit can resend it later and have it processed again exactly as if it were new — demonstrated above by the identical request being accepted twice. Adding a unique nonce and a timestamp, both covered by the same MAC, means the server can now distinguish 'a message I have already processed' (nonce already seen) and 'a message sent too long ago to still be trusted' (stale timestamp) from a genuinely new request, closing the replay window.

Evaluation of limitations

- Requires server-side state: the server must remember every nonce seen within the validity window, which grows with traffic volume and needs periodic cleanup of expired entries.
- Requires reasonably synchronized clocks between client and server; excessive clock skew can cause legitimate requests to be rejected as 'stale', or (if the window is set too generously to compensate) can widen the replay window unnecessarily.
- Does not stop a real-time relay/MITM attack: if an attacker forwards a captured request immediately (within the validity window, before the legitimate copy arrives), the nonce/timestamp check alone will not catch it — this requires additional defenses such as mutual authentication, TLS channel binding, or challenge-response protocols.

Experiment 5: Performance Comparison of Digital Signature Algorithms

RSA-2048 (PSS padding) and ECDSA over the P-256 curve were benchmarked over 20 trials each for key generation, signing, and verification, using the cryptography library:

```
# RSA
priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sig  = priv.sign(MESSAGE, padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                 salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256())

# ECDSA (P-256)
priv = ec.generate_private_key(ec.SECP256R1())
sig  = priv.sign(MESSAGE, ec.ECDSA(hashes.SHA256()))
```

Results

Metric	RSA-2048	ECDSA (P-256)
Key generation (avg)	58.06 ms	0.153 ms
Signing (avg)	1.359 ms	0.675 ms
Verification (avg)	0.099 ms	0.114 ms
Signature size	256 bytes	71 bytes

ECDSA key generation was roughly 380× faster than RSA-2048 in this benchmark, and produced a signature about 3.6× smaller.

Analysis: key generation, signing, and verification efficiency

RSA key generation is by far the most expensive operation measured, because it requires searching for two large random primes and testing them for primality — an inherently slow process. ECDSA key generation, by contrast, is just a single random scalar multiplication on the elliptic curve, which is why it is orders of magnitude faster. Signing is also faster for ECDSA in this test. RSA verification, however, is slightly faster than ECDSA verification, because RSA verification uses a small public exponent (e = 65537) requiring very few modular multiplications, whereas ECDSA verification requires two scalar multiplications on the curve.

Evaluation: suitability for different real-world applications

RSA: well suited to systems where verification happens far more often than signing or key generation (e.g., software update verification, long-lived certificates), and where broad legacy compatibility matters — RSA is universally supported by older systems and libraries.

ECDSA/EdDSA: far better suited to environments where key generation and signing happen frequently and resources are constrained — mobile devices, IoT hardware, high-throughput TLS handshakes, and blockchain/cryptocurrency systems — because of its much lower computational cost and smaller key/signature sizes, which also reduce bandwidth and storage overhead. This is why modern protocols (TLS 1.3, SSH, most new deployments) default to elliptic-curve signatures over RSA wherever legacy compatibility is not a constraint.